

EXPERIMENT-1:

Aim:

To capture and analyze the TCP Three-Way Handshake using Wireshark.

Procedure:

1. Open Wireshark.
2. Select the active Wi-Fi interface and start packet capture.
3. Open a web browser and visit a website (e.g., <https://google.com>).
4. Stop the packet capture after the webpage loads.
5. Apply the display filter:

`tcp.stream == 9`

(Your stream number is **9**.)

6. Observe the TCP packets containing the **SYN**, **SYN-ACK**, and **ACK** flags.

Observation:

The captured TCP connection contains the following three packets:

- **Packet 711:** SYN – The client (172.25.55.66) initiates the connection by sending a SYN packet to the server (52.123.253.73).
- **Packet 744:** SYN, ACK – The server responds with a SYN-ACK packet, acknowledging the client's request and synchronizing its own sequence number.
- **Packet 746:** ACK – The client sends an ACK packet to acknowledge the server's response, completing the TCP connection establishment.

Role of Each Flag

1. SYN (Synchronize)

- Initiates the TCP connection.
- Synchronizes the client's initial sequence number.
- Sent by the client.

2. SYN-ACK (Synchronize + Acknowledge)

- Sent by the server.
- Acknowledges the client's SYN packet.
- Synchronizes the server's sequence number.

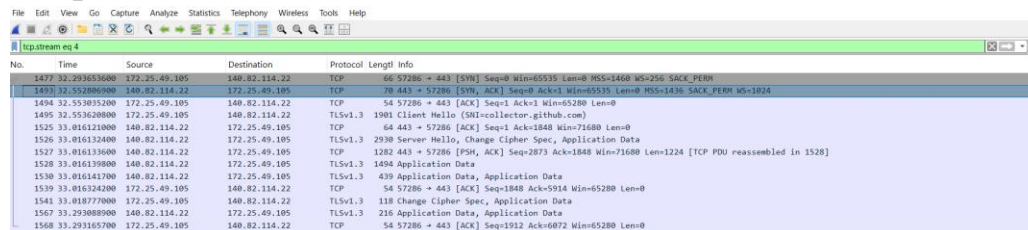
3. ACK (Acknowledge)

- Sent by the client.
- Acknowledges the server's SYN-ACK packet.
- Completes the three-way handshake.

Explanation:

The TCP Three-Way Handshake is used to establish a reliable communication session between a client and a server. First, the client sends a SYN packet requesting a connection. The server replies with a SYN-ACK packet, confirming the request and sending its own synchronization information. Finally, the client responds with an ACK packet, confirming receipt of the server's response. Once these three packets are exchanged, the connection is established, and reliable data transfer can begin.

Output:



The image shows a Wireshark packet capture of a TCP connection. The packet list on the left shows several packets, with packet 1493 selected. The packet details pane on the right shows the structure of the selected packet, which is a TCP Reset (RST) with sequence number 5535 and window size 0. The packet bytes pane at the bottom shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
1477	32.293653600	172.25.49.105	140.82.114.22	TCP	66	57286 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1493	32.552000000	140.82.114.22	172.25.49.105	TCP	70	443 → 57286 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1436 SACK_PERM WS=1824
1494	32.553035200	172.25.49.105	140.82.114.22	TCP	54	57286 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1495	32.553620000	172.25.49.105	140.82.114.22	TLSv1.3	1901	Client Hello [SHA256]
1525	33.016121000	140.82.114.22	172.25.49.105	TCP	64	443 → 57286 [ACK] Seq=1 Ack=1848 Win=71680 Len=0
1526	33.016132400	140.82.114.22	172.25.49.105	TLSv1.3	2930	Server Hello, Change Cipher Spec, Application Data
1527	33.016139600	140.82.114.22	172.25.49.105	TCP	1282	443 → 57286 [PSH, ACK] Seq=2873 Ack=1848 Win=71680 Len=1224 [TCP PDU reassembled in 1528]
1528	33.016139800	140.82.114.22	172.25.49.105	TLSv1.3	1404	Application Data
1530	33.016141700	140.82.114.22	172.25.49.105	TLSv1.3	439	Application Data
1539	33.016324200	172.25.49.105	140.82.114.22	TCP	54	57286 → 443 [ACK] Seq=1848 Ack=5914 Win=65280 Len=0
1541	33.018777000	172.25.49.105	140.82.114.22	TLSv1.3	118	Change Cipher Spec, Application Data
1567	33.293080900	140.82.114.22	172.25.49.105	TLSv1.3	216	Application Data
1568	33.293105700	172.25.49.105	140.82.114.22	TCP	54	57286 → 443 [ACK] Seq=1912 Ack=6072 Win=65280 Len=0

Conclusion:

The Wireshark capture successfully demonstrates the TCP Three-Way Handshake using the **SYN**, **SYN-ACK**, and **ACK** packets. This process establishes a reliable connection, synchronizes sequence numbers, and ensures that both the client and server are ready for secure and reliable communication.

EXPERIMENT-2

Aim:

To capture a DNS query using UDP in Wireshark, compare the UDP header size with TCP, and observe the absence of sequence numbers and acknowledgments.

Procedure:

1. Open Wireshark.
2. Select the active Wi-Fi interface.
3. Start packet capture.
4. Open Command Prompt and execute:

```
nslookup google.com
```

5. Stop the packet capture.
6. Apply the display filter:

```
udp.port == 53
```

7. Select the DNS query or response packet.
8. Expand the **User Datagram Protocol (UDP)** and **Domain Name System (DNS)** sections to view the packet details.

Observation:

The captured packet shows that DNS communication uses the **UDP protocol** over **port 53**. The UDP header contains only the following fields:

- Source Port
- Destination Port
- Length
- Checksum

Unlike TCP, the UDP header **does not contain Sequence Number or Acknowledgment Number fields**, making it much simpler and smaller.

UDP vs TCP Header Comparison

Feature	UDP	TCP
Header Size	8 bytes	20 bytes (minimum)
Source Port	Yes	Yes
Destination Port	Yes	Yes
Sequence Number	No	Yes
Acknowledgment Number	No	Yes
Checksum	Yes	Yes
Reliability	No	Yes
Connection Type	Connectionless	Connection-oriented
Speed	Faster	Slower

Explanation:

The Wireshark capture confirms that DNS uses **UDP** because DNS messages are generally small and require fast transmission. UDP has a fixed **8-byte header**, while TCP has a minimum **20-byte header**. Since UDP does not establish a connection or maintain sequence numbers and acknowledgments, it reduces communication overhead and allows DNS queries to be completed quickly. However, because UDP does not retransmit lost packets or guarantee delivery, it is less reliable than TCP.

Impact on Speed and Reliability

Speed

- Small **8-byte header**.
- No connection establishment.
- No acknowledgments.
- Low protocol overhead.
- Faster transmission and lower latency.

Reliability

- No sequence numbers.
- No acknowledgments.
- No retransmission of lost packets.
- Packets may be lost or arrive out of order.
- Less reliable than TCP.

Output:

The image shows a Wireshark packet capture of a DNS response. The packet list on the left shows a single packet (No. 2578) of type 'Domain Name System (response)' with a length of 139 bytes. The packet details pane shows the following structure:

- Frame 2578: Packet, 139 bytes on wire (1112 bits), 139 bytes captured (1112 bits) on interface \Device\NPF_{292CBC24-BA1F-4DB6-B0F4-D329E9BF7EF5}, id 0
- Ethernet II, Src: Cisco_gdrc0:d7 (24:d5:e4:6d:c0:d7), Dst: Intel_51:d4:7f (b8:f7:75:51:d4:7f)
- Internet Protocol Version 4, Src: 172.21.100.31, Dst: 172.25.49.105
- User Datagram Protocol, Src Port: 53, Dst Port: 56866
- Domain Name System (response)

The packet bytes pane shows the raw data in hexadecimal and ASCII:

```

0000  b8 f7 75 51 d4 7f 24 d5 e4 6d c0 d7 08 00 45 00  --uQ..$.m...E-
0010  00 7d 01 86 00 00 7e 11 4d 33 ac 15 64 1f ac 19  --}....MD..d...
0020  31 69 00 35 db 02 00 69 fa f5 7b e2 85 83 00 01  11 5...i...{....
0030  00 00 00 01 00 00 04 77 70 61 64 08 73 61 76 65  ....w pod: save
0040  65 74 68 61 83 6e 65 74 00 00 01 00 01 00 73 61  etha net .....sa
0050  76 65 65 74 68 61 83 6e 65 74 00 00 06 00 01 00  veetha n et.....
0060  00 0e 10 00 26 02 73 33 c0 23 0a 68 6f 73 74 6d  ...&.s #.hostm
0070  61 73 74 65 72 c0 23 00 00 43 72 00 00 03 04 00  aster # -Cr.....
0080  00 02 58 00 01 51 80 00 00 03 84  --X.X.Q...

```

The packet info pane at the bottom shows: No. 2578 - Time: 0.244458000 - Source: 172.21.100.31 - Destination: 172.25.49.105 - Protocol: DNS - Length: 139 - Info: Standard query response 0x7be2 has such name A record.sveetha.net SGA 13.sveetha.net

Conclusion:

The experiment demonstrates that **DNS typically uses UDP** because it is lightweight and efficient. The absence of sequence numbers and acknowledgments, along with its small **8-byte header**, allows faster communication. However, this design sacrifices reliability compared to TCP, making UDP suitable for applications where speed is more important than guaranteed delivery.

EXPERIMENT-3

Aim:

To simulate packet loss, observe TCP retransmission in Wireshark, and compare TCP with UDP in terms of reliability and speed.

Procedure:

1. Open **Wireshark**.
2. Select the active **Wi-Fi** interface and start packet capture.
3. Open a website or start downloading a file.
4. Briefly disconnect the Wi-Fi connection for **5–10 seconds**.
5. Reconnect the Wi-Fi.
6. Stop the packet capture.
7. Apply the display filter:

tcp

8. Observe packets marked as:
 - **TCP Retransmission**
 - **TCP Spurious Retransmission**
 - **TCP Dup ACK**
9. Apply the filter:

udp

10. Observe that UDP packets are **not retransmitted**.

Observation:

The Wireshark capture shows several **TCP Retransmission**, **TCP Spurious Retransmission**, and **TCP Duplicate ACK** packets after the temporary network interruption. These packets indicate that some data packets were lost during transmission. Since the sender did not receive the expected acknowledgment, TCP retransmitted the missing packets. When observing UDP traffic, no retransmission packets were found because UDP does not provide a mechanism for detecting or recovering lost packets.

TCP vs UDP During Packet Loss

Feature	TCP	UDP
Detects Packet Loss	Yes	No
Retransmits Lost Packets	Yes	No
Uses Sequence Numbers	Yes	No
Uses Acknowledgments	Yes	No
Reliability	High	Low
Speed	Slower	Faster

Explanation:

TCP is a connection-oriented protocol that ensures reliable communication. It uses **sequence numbers** to track transmitted packets and **acknowledgments (ACKs)** to confirm successful delivery. If an acknowledgment is not received within a certain time, TCP assumes that the packet has been lost and retransmits it. This guarantees reliable delivery but increases communication delay and network overhead.

UDP is a connectionless protocol that does not use acknowledgments, sequence numbers, or retransmissions. If a UDP packet is lost, it is simply discarded without being resent. This reduces overhead and provides faster communication but does not guarantee reliable delivery.

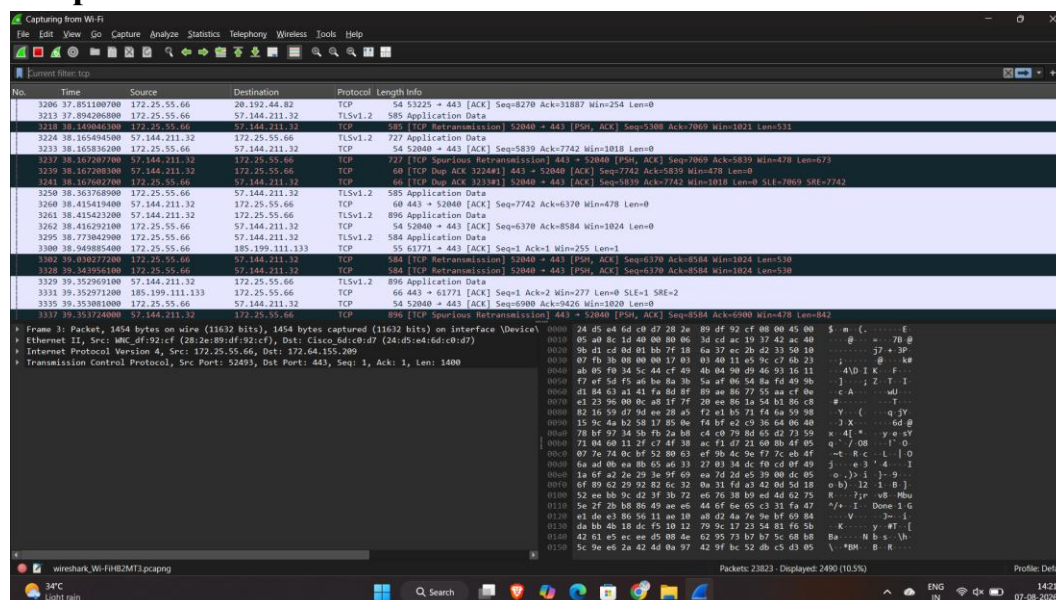
Why TCP is Reliable but Slower

- Uses acknowledgments to confirm delivery.
- Detects lost packets.
- Retransmits missing packets.
- Maintains packet order using sequence numbers.
- Introduces extra delay due to retransmissions and error recovery.

Why UDP is Faster but Less Reliable

- No connection setup.
- No acknowledgments.
- No retransmissions.
- Smaller protocol overhead.
- Lost packets are not recovered.

Output:



Conclusion:

The Wireshark capture demonstrates that **TCP retransmits lost packets** after detecting packet loss through duplicate acknowledgments and timeout mechanisms. This makes TCP highly reliable for applications requiring accurate data delivery. In contrast, **UDP does not retransmit lost packets**, making it faster but less reliable. Therefore, TCP is preferred for

applications such as web browsing and file transfer, while UDP is suitable for real-time applications such as video streaming, online gaming, and voice communication.

EXPERIMENT-4:

Aim:

To capture DNS traffic using Wireshark while resolving a domain name, identify the DNS query type and response, measure the DNS resolution time, and explain why DNS typically uses UDP.

Procedure:

1. Open **Wireshark**.
2. Select the active **Wi-Fi** interface.
3. Start packet capture.
4. Open Command Prompt and execute:

nslookup google.com

5. Stop the packet capture.
6. Apply the display filter:

dns

7. Observe the DNS query and DNS response packets.
8. Identify the query type (**A**, **AAAA**, or **MX**) and the returned response.
9. Measure the time taken between the query and the response.

Observation:

The Wireshark capture shows DNS queries and their corresponding responses.

Query Type A

Field	Value
Domain Name	google.com
Query Type	A (IPv4 Address)
Response	142.250.182.78

Query Type AAAA

Field	Value
Domain Name	google.com
Query Type	AAAA (IPv6 Address)
Response	2404:6800:4007:835::200e

DNS Resolution Time

From your capture:

- A Query Time: 14.180772300 s
- A Response Time: 14.486452600 s

Resolution Time = Response Time – Query Time

= 14.486452600 – 14.180772300

= 0.3056803 seconds

≈ 306 ms

Why DNS Typically Uses UDP

DNS generally uses **UDP** because:

- It is **connectionless**, so there is no connection setup.

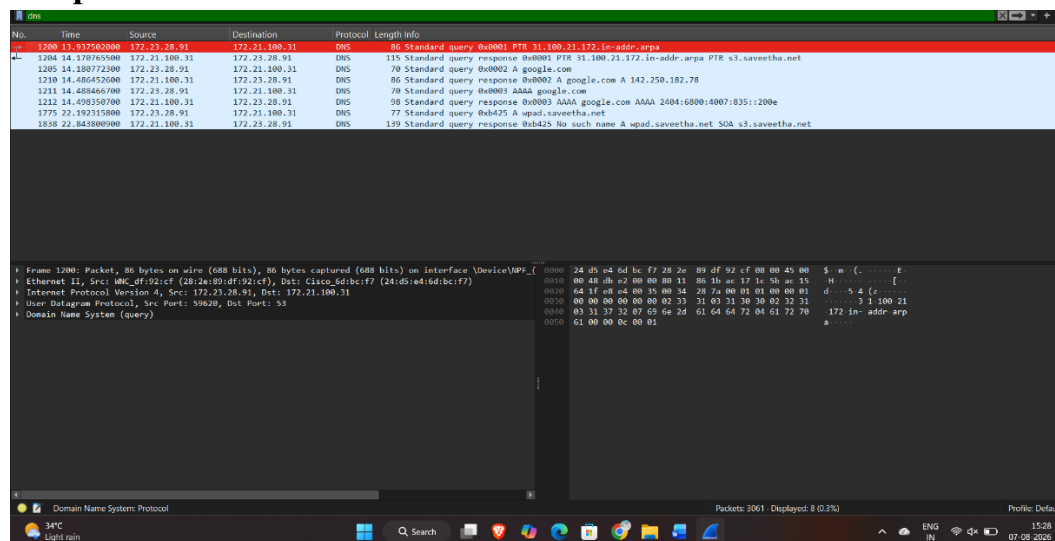
- It has a small **8-byte header**, reducing overhead.
- DNS requests and responses are usually small.
- It provides **fast communication** with low latency.
- It reduces network traffic and improves efficiency.

When DNS responses are too large or for operations such as zone transfers, **TCP** is used instead.

Explanation:

The Wireshark capture shows that the client sends a DNS query to the DNS server requesting the IP address of **google.com**. The DNS server responds with the requested record. In this capture, both an **A record** (IPv4 address) and an **AAAA record** (IPv6 address) are returned. The time difference between the query and the response represents the DNS resolution time. DNS uses UDP because it is lightweight, connectionless, and provides fast name resolution with minimal overhead.

Output:



Conclusion:

The experiment successfully captured DNS traffic while resolving **google.com**. The DNS server returned an **A record (142.250.182.78)** and an **AAAA record (2404:6800:4007:835::200e)**. The measured DNS resolution time was approximately **306 ms**. DNS typically uses **UDP** because it offers faster communication with lower overhead, making it suitable for quick domain name resolution.